

BASES TECNOLÓGICAS PARA EL SERVICIO

MÓDULO: BASES DE DATOS

ACTIVIDAD: Reporte de la práctica
Gestión de Bases de Datos

ALUMNO: CESAR CEDILLO
RODRÍGUEZ

GRUPO: 8

DOCENTE: JUAN MANUEL
TOLENTINO MORALES

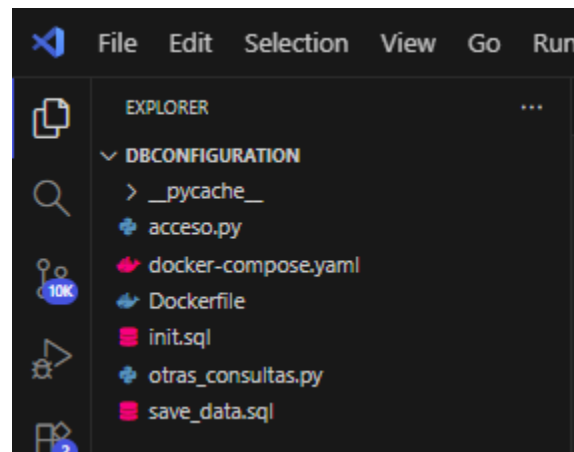
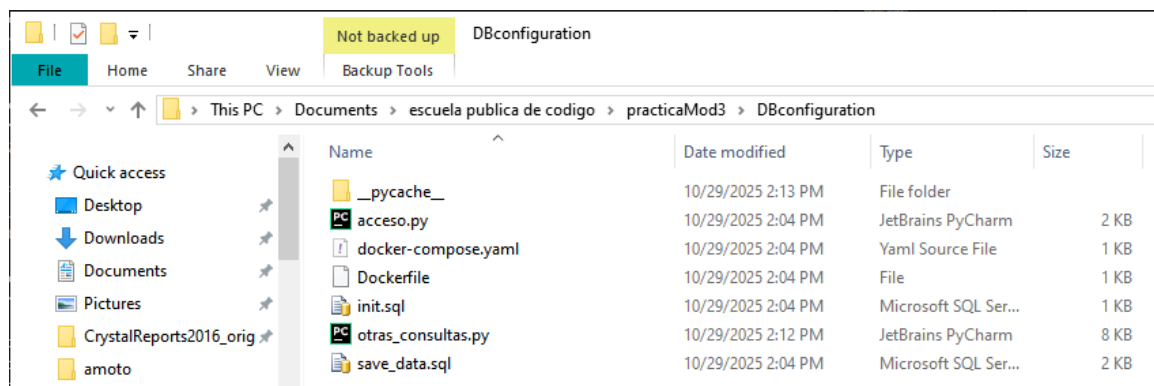
1. INTRODUCCIÓN Y OBJETIVO

Durante esta fase se configuró un entorno de base de datos PostgreSQL mediante contenedores Docker, se definió la estructura de las tablas requeridas en la práctica, se corrigieron los nombres conforme a las preguntas de reforzamiento, y se elaboró un script en Python para registrar nuevos usuarios en la base de datos.

Objetivo general

Implementar un entorno de base de datos PostgreSQL mediante contenedores Docker, diseñar la estructura de tablas de un sistema de gestión de usuarios y puestos de trabajo, insertar datos de prueba, y desarrollar un script en Python para registrar nuevos usuarios.

2. CONFIGURACIÓN DEL ENTORNO



El contenedor se configuró en el archivo docker-compose.yaml para usar el **puerto 5433** (ya que el 5432 estaba ocupado) y el usuario administrador Admin.

El volumen postgres_data_mod3 permite conservar la información incluso si el contenedor se detiene.

```

docker-compose.yml
1  services:
2    postgres:
3      build: .
4      container_name: ContDBpractMod3
5      ports:
6        - "5433:5432"
7      environment:
8        POSTGRES_USER: Admin
9        POSTGRES_PASSWORD: p4sswOrdDB
10       POSTGRES_DB: gestion_usuarios
11       volumes:
12         - postgres_data_mod3:/var/lib/postgresql/data
13
14  volumes:
15    postgres_data_mod3:
16      driver: local
17

```

[Containers](#) / ContDBpractMod3


11c94f66380f
dbconfiguration-r

ContDBpractMod3

STATUS
 Running (0 seconds ago)






5433:5432

3. CREACIÓN DE LA BASE DE DATOS Y DE LAS TABLAS

Los scripts init.sql y save_data.sql fueron modificados conforme a las preguntas del **ejercicio de reforzamiento**.

Respuesta a la pregunta 2

¿Por qué renombrar la tabla credenciales a credenciales_usuarios?

Porque en la base de datos también existe la tabla usuarios, y usar nombres más específicos mejora la claridad y evita ambigüedad en las consultas y las relaciones.

Respuesta a la pregunta 3

Agrega la tabla de puestos de trabajo y relaciónala con usuarios.

Se creó la tabla puestos con campos:

id_puesto, nombre, descripcion, salario_base, activo, fecha_alta.

Luego se añadió la relación usuarios.puesto_id → puestos.id_puesto.

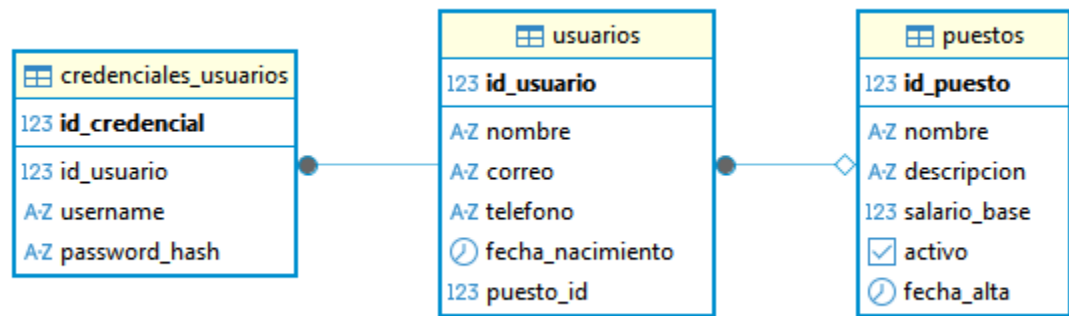
En el archivo init.sql se detallan las siguientes tablas:

- **usuarios** → información general del personal.
- **credenciales_usuarios** → credenciales vinculadas a cada usuario.

- **puestos** → catálogo de puestos de trabajo con su salario y descripción.

Además, se establecieron las claves foráneas correspondientes:

- `credenciales_usuarios.id_usuario` → `usuarios.id_usuario`
- `usuarios.puesto_id` → `puestos.id_puesto`



5. INSERCIÓN DE LOS DATOS INICIALES

El archivo `save_data.sql` contiene datos de prueba para las tres tablas:

- Se agregaron **tres puestos** (“Auxiliar administrativo”, “Técnico especializado”, “Analista de datos”).
- Se insertaron **cinco usuarios** asignados a distintos puestos.
- Se registraron sus **credenciales** con identificadores de usuario

Script SQL:

```
SELECT * FROM usuarios;
SELECT * FROM puestos;
```

Tabla usuarios 1:

	123 id	A-Z nombre	A-Z correo	A-Z telef	fecha	123 puesto_id
1	1	Juan Pérez	juan.perez1@example.com	1234567890	1985-01-15	2
2	2	Ana Gómez	ana.gomez2@example.com	1234567891	1990-03-22	1
3	3	Luis Martínez	luis.martinez3@example.com	1234567892	1988-07-10	3
4	4	Maria López	maria.lopez4@example.com	1234567893	1992-11-05	1
5	5	Carlos Ruiz	carlos.ruiz5@example.com	1234567894	1980-06-25	2
6	6	Nuevo Usuario	nuevo.usuario@example.com	5551234567	1990-01-01	1

SELECT * FROM puestos p Enter a SQL expression to filter results (use Ctrl+Space)							
Grid	123	AZ nombre	AZ descripcion	123 sal	[v]	fecha	
Text	1	1	Auxiliar administrativo	Tareas administrativas y atención a usuari	8,000	[v]	2025-10-29
	2	2	Técnico especializado	Mantenimiento y soporte técnico	12,000	[v]	2025-10-29
	3	3	Analista de datos	Procesamiento y análisis de información	15,000	[v]	2025-10-29

REGISTRO DE NUEVOS USUARIOS

Crea una función en Python para registrar usuarios.

El archivo acceso.py permite conectar al contenedor, insertar nuevos registros y generar el hash de la contraseña con **SHA-256**.

La función registrar_usuario(nombre, correo, telefono, fecha_nacimiento, username, password, puesto_id) inserta automáticamente los datos en las tablas usuarios y credenciales_usuarios.

```
if __name__ == "__main__":
    success, msg = registrar_usuario(
        'Manuel', 'manuel@epc.com', '5551234567', '1989-01-01', 'manuel.user', 'miContraseñaDificil', 2
    )
    print(success, msg)
```

```
PS C:\Users\César\Documents\escuela publica de codigo\practicaMod3\DBconfiguration> & C:\Users\César\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/César/Documents/escuela publica de codigo/practicaMod3\DBconfiguration/acceso.py"
True Usuario registrado con id 11
PS C:\Users\César\Documents\escuela publica de codigo\practicaMod3\DBconfiguration>
```

SELECT * FROM usuarios u Enter a SQL expression to filter results (use Ctrl+Space)							
Grid	123	AZ nombre	AZ correo	AZ telefono	feh	123 puesto_id	
Text	1	1	Juan Pérez	juan.perez1@example.com	1234567890	1985-01-15	2
	2	2	Ana Gómez	ana.gomez2@example.com	1234567891	1990-03-22	1
	3	3	Luis Martínez	luis.martinez3@example.com	1234567892	1988-07-10	3
	4	4	Maria López	maria.lopez4@example.com	1234567893	1992-11-05	1
	5	5	Carlos Ruiz	carlos.ruiz5@example.com	1234567894	1980-06-25	2
	6	6	Nuevo Usuario	nuevo.usuario@example.com	5551234567	1990-01-01	1
	7	11	Manuel	manuel@epc.com	5551234567	1989-01-01	2

SELECT * FROM credenciales_usuario Enter a SQL expression to filter results (use Ctrl+Space)				
Grid	123 id_credencial	123 id_usuario	AZ usernam	AZ password_hash
Text	1	1	juan.perez1	hash_juan_perez
	2	2	ana.gomez2	hash_ana_gomez
	3	3	luis.martinez3	hash_luis_martinez
	4	4	maria.lopez4	hash_maria_lopez
	5	5	carlos.ruiz5	hash_carlos_ruiz
	6	6	nuevo.user	afc125e557d1be4ad0050e9094f07331cd70a4ed3d98947
	7	8	manuel.user	0db559cb2fd280e8260dc1d45d7bdcd5c8ebbf7a3afa

puestos 1

SELECT * FROM puestos p

	123 id	AZ nombre	AZ descripcion	123	act	fecha_alta
1	1	Auxiliar administrativo	Tareas administrativas y atención a usuari	8,000	[v]	2025-10-29
2	2	Técnico especializado	Mantenimiento y soporte técnico	12,000	[v]	2025-10-29
3	3	Analista de datos	Procesamiento y análisis de información	15,000	[v]	2025-10-29

CONSULTAS Y VALIDACIÓN DE USUARIOS

Para complementar el registro de nuevos usuarios, se desarrolló el script **consultas.py**, cuya finalidad es **verificar credenciales y recuperar los datos de un usuario registrado** en la base de datos PostgreSQL.

Este script establece una conexión al mismo contenedor Docker de PostgreSQL, solicita las credenciales mediante consola, y realiza una consulta segura usando el hash SHA-256 de la contraseña ingresada.

El programa compara los valores proporcionados con los registros almacenados en la tabla `credenciales_usuarios`. Si el usuario y la contraseña coinciden, se muestran los datos asociados en la tabla `usuarios`; en caso contrario, se indica que las credenciales son incorrectas.

El código implementa las siguientes funciones principales:

- **conectar_db()**: establece la conexión con la base de datos.
- **hash_password()**: genera el hash SHA-256 de la contraseña ingresada.
- **obtener_datos_usuario()**: ejecuta la consulta SQL y muestra la información del usuario si las credenciales son válidas.

```
PS C:\Users\César\Documents\escuela publica de codigo\practicaMod3\DBconfiguration> python consulta.py
Inicio de sesión en la base de datos
Ingrese su usuario: manuel.user
Ingrese su contraseña (VISIBLE): miContraseñaDificil

Datos del usuario encontrado:
ID: 11
Nombre: Manuel
Correo: manuel@epc.com
Teléfono: 5551234567
Fecha de Nacimiento: 1989-01-01
PS C:\Users\César\Documents\escuela publica de codigo\practicaMod3\DBconfiguration>
```

CONCLUSIONES

Durante el desarrollo de la práctica se logró implementar satisfactoriamente un entorno de base de datos PostgreSQL dentro de un contenedor Docker, garantizando su portabilidad y fácil despliegue. La estructura relacional diseñada cumple con los requerimientos establecidos en el ejercicio de reforzamiento, al incluir las tablas de usuarios, credenciales y puestos, correctamente vinculadas mediante claves foráneas. Además, el script desarrollado en Python permitió comprobar la integración correcta entre la aplicación y la base de datos, logrando así el registro de nuevos usuarios y aplicando medidas de seguridad como el uso de hash para las contraseñas. En conjunto, esta práctica demostró el funcionamiento completo de un sistema de gestión de datos funcional, y reutilizable.

*Nota, se cargaron solo los primeros 5 usuarios del ejercicio, se agrego el uso de hash y se separo el acceso de las consultas